

LAPORAN TEORI

PENGOLAHAN CITRA DIGITAL



NAMA : Ahmad Miftahul Arza Firisky

NIM : 202331067

KELAS : A

DOSEN : Dr. Dra. Dwina Kuswardani, M.Kom

NO.PC : 12

ASISTEN : 1. Clarenca Sweetdiva Pereira

2. Viana Salsabila Fairuz Syahla

3. Kashrina Masyid Azka

4. Sasikirana Ramadhanty Setiawan Putri

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2025

1. Jelaskan apa itu operasi aras titik!

Operasi aras titik adalah operasi pengolahan citra yang hanya dilakukan pada satu piksel tunggal dalam citra. Artinya, transformasi atau perubahan hanya bergantung pada nilai intensitas dari satu piksel tersebut, tanpa mempertimbangkan piksel di sekitarnya. Operasi ini dapat berupa operasi linier atau non-linier, misalnya operasi penambahan konstan, pengurangan, atau transformasi logaritmik.

2. Jelaskan apa itu operasi pengambangan (thresholding) dan berikan contohnya!

Operasi pengambangan (thresholding) adalah proses mengubah nilai intensitas piksel menjadi hanya dua nilai (biasanya hitam atau putih), berdasarkan ambang batas tertentu (threshold TTT). Contoh :

- Jika intensitas piksel < 128 , maka piksel diubah menjadi hitam (0).
 - Jika intensitas piksel ≥ 128 , maka piksel diubah menjadi putih (1).
- Operasi ini biasa digunakan untuk mengubah citra ke bentuk biner, yang sangat berguna untuk segmentasi atau ekstraksi objek.

3. Jelaskan perbedaan dari citra biner dan citra negatif!

- Citra Biner : Merupakan citra yang hanya memiliki dua tingkat intensitas, yaitu 0 dan 1 (atau 0 dan 255 untuk representasi dalam 8-bit), biasanya hasil dari proses thresholding. Citra ini menampilkan hanya dua warna, biasanya hitam dan putih.
- Citra Negatif : Diperoleh dengan mengubah setiap nilai intensitas piksel ke nilai komplementernya. Untuk citra 8-bit (dengan skala keabuan 0–255).

4. Jelaskan proses peningkatan kecerahan yang sudah di praktikumkan

Proses peningkatan kecerahan dilakukan dengan menambahkan nilai konstan b ke setiap piksel.

Rumus:

$$f(x,y)' = f(x,y) + b$$

Jika nilai b positif, kecerahan meningkat. Jika negatif, kecerahan berkurang. Nilai hasil juga perlu di clipping agar tetap berada dalam rentang 0–255 (untuk citra 8-bit).

5. Jelaskan arti histogram setelah operasi peningkatan kecerahan dari citra asli yang sudah dipraktikumkan!

Histogram citra menunjukkan distribusi intensitas piksel dalam citra. Setelah operasi peningkatan kecerahan, histogram akan bergeser ke kanan artinya intensitas piksel secara keseluruhan meningkat.

Jadi area gelap dalam histogram akan berkurang dan area terang bertambah. Ini menunjukkan bahwa citra secara visual menjadi lebih terang. Pergeseran ini juga dapat membuat kontras antar objek menjadi lebih jelas jika dilakukan dengan benar.

LAPORAN PRAKTIKUM

PENGOLAHAN CITRA DIGITAL



NAMA : Ahmad Miftahul Arza Firisky

NIM : 202331067

KELAS : A

DOSEN : Dr. Dra. Dwina Kuswardani, M.Kom

NO.PC : 12

ASISTEN : 1. Clarenca Sweetdiva Pereira

2. Viana Salsabila Fairuz Syahla

3. Kashrina Masyid Azka

4. Sasikirana Ramadhanty Setiawan Putri

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2025

Mengolah gambar yang sama dengan gambar laporan 1 menggunakan metode-metode yang telah di praktikkan. sebelumnya (praktikum 3).

1. Import Library

Import Library

```
[2]: import cv2
import matplotlib.pyplot as plt
import numpy as np
#202331067_Ahmad Miftahul Arza Firisky
```

cv2 adalah pustaka dari OpenCV yang digunakan untuk memproses gambar dan video. matplotlib.pyplot digunakan untuk menampilkan grafik atau gambar, terutama dalam konteks visualisasi hasil pengolahan citra. numpy berfungsi untuk menangani operasi numerik, terutama yang berkaitan dengan array, karena citra digital direpresentasikan dalam bentuk array/matriks.

2. Membaca Data Gambar

Membaca Data Gambar

```
[4]: img = cv2.imread('kanguru1.png')
#202331067_Ahmad Miftahul Arza Firisky
```

```
[5]: img.shape
#202331067_Ahmad Miftahul Arza Firisky
```

```
[5]: (355, 445, 3)
```

```
[6]: [baris, kolom] = img.shape[:2]
#202331067_Ahmad Miftahul Arza Firisky
```

kanguru1.png di gunakan untuk membaca sebuah gambar berkas bernama 'kanguru1.png' menggunakan fungsi cv2.imread(), dan menyimpannya dalam variabel img. Fungsi ini memuat gambar sebagai array NumPy tiga dimensi, di mana setiap elemen array merepresentasikan nilai piksel pada posisi tertentu dalam citra.

Selanjutnya, img.shape digunakan untuk mendapatkan dimensi gambar tersebut, yaitu tinggi jumlah baris, lebar jumlah kolom, dan jumlah warnanya 3 warna untuk RGB. kode img.shape[:2] digunakan untuk mengambil dua nilai pertamanya yaitu jumlah baris dan kolom yang kemudian disimpan masing-masing ke dalam variabel baris dan kolom.

3. Convert BGR to RGB

Convert BGR to RGB

```
[8]: rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
#202331067_Ahmad Miftahul Arza Firisky
```

`rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` digunakan untuk mengubah format warna gambar dari BGR ke RGB. Fungsi `cv2.cvtColor()` ini yang melakukan proses konversi warna, dan hasilnya disimpan dalam variabel `rgb`. Setelah dikonversi, gambar siap untuk ditampilkan dengan benar melalui fungsi plotting seperti `plt.imshow()`.

4. Menampilkan Histogram

Menampilkan Histogram

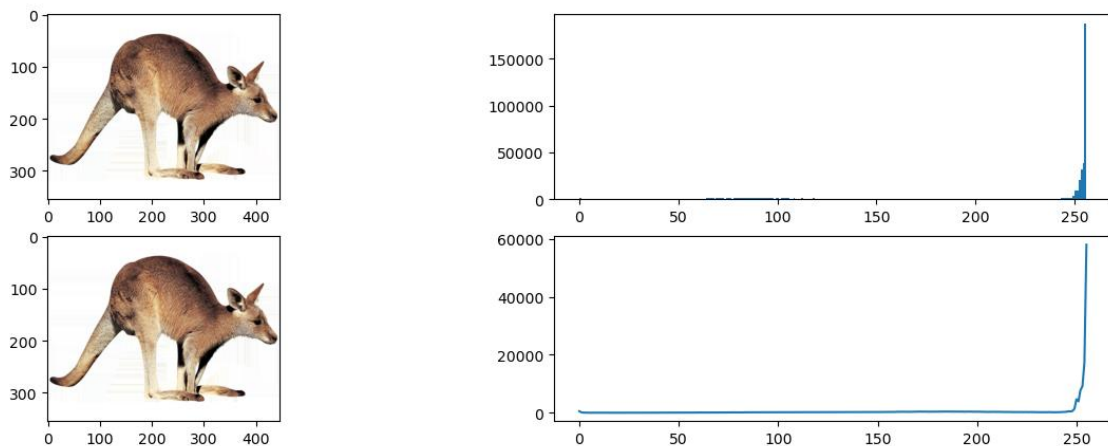
```
[10]: fig, axs = plt.subplots(2,2, figsize = (15,5))

#Cara 1
axs[0,0].imshow(rgb)
axs[0,1].hist(rgb.ravel(),256,[0,256])

#Cara 2
hist = cv2.calcHist([rgb],[0], None,[256],[0,256])
axs[1,0].imshow(rgb)
axs[1,1].plot(hist)

#Menampilkan
plt.show()
#202331067_Ahmad Miftahul Arza Firisky
```

Output :



Pertama, `fig, axs = plt.subplots(2,2, figsize = (15,5))` membuat kanvas gambar dengan 2 baris dan 2 kolom, sehingga total terdapat 4 area subplot untuk menampilkan hasil visualisasi. Ukuran keseluruhan figur diatur menjadi 15x5 inci agar hasil tampak jelas.

- Cara 1:

- `axs[0,0].imshow(rgb)` menampilkan gambar berformat RGB di subplot kiri atas.
- `axs[0,1].hist(rgb.ravel(), 256, [0,256])` membuat histogram dari keseluruhan piksel RGB yang sudah diratakan ke dalam satu dimensi (menggunakan `.ravel()`), sehingga seluruh nilai intensitas dari ketiga kanal digabung menjadi satu. Histogram ini menunjukkan berapa banyak piksel yang memiliki intensitas tertentu (0–255).

- Cara 2 :

- `cv2.calcHist([rgb],[0], None,[256],[0,256])` menghitung histogram citra dengan menggunakan OpenCV. Di sini hanya kanal ke-0 (biasanya biru pada BGR, tapi karena sudah dikonversi ke RGB, ini menjadi kanal merah) yang dihitung. Hasil histogram ini kemudian ditampilkan menggunakan `axs[1,1].plot(hist)`.

- `axs[1,0].imshow(rgb)` kembali menampilkan gambar RGB, kali ini di subplot kiri bawah, agar selaras dengan histogram yang muncul di sebelah kanannya.

Dan akhirnya `plt.show()` digunakan untuk menampilkan semua subplot dalam satu tampilan.

5. Operasi Piksel Untuk Meningkatkan Kecerahan

Operasi Piksel Untuk Meningkatkan Kecerahan

```
[12]: beta = 40
citra_cerah = np.zeros((baris, kolom, 3))

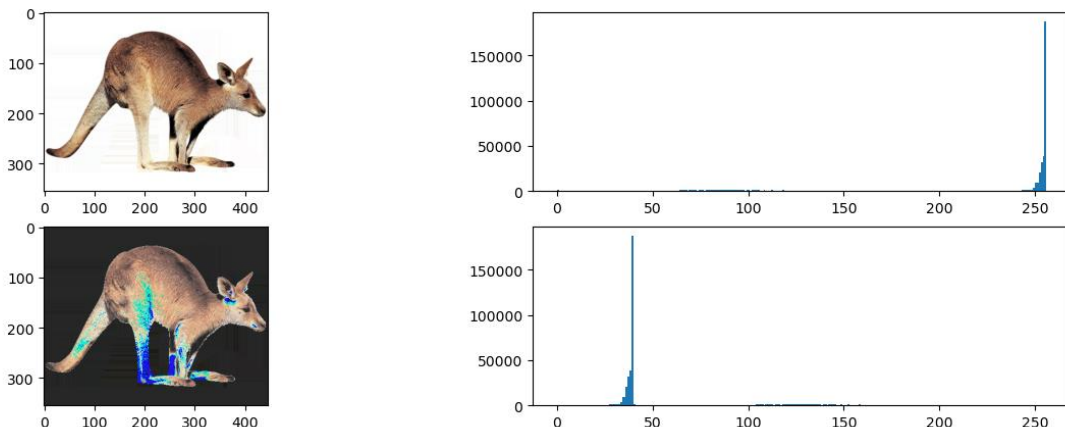
for x in range(baris):
    for y in range(kolom):
        gx = rgb[x,y] + beta
        citra_cerah[x,y] = gx

citra_cerah = citra_cerah.astype(np.uint8)

fig, axs = plt.subplots(2,2, figsize=(15,5))
axs[0,0].imshow(rgb)
axs[0,1].hist(rgb.ravel(),256, [0,256])
axs[1,0].imshow(citra_cerah)
axs[1,1].hist(citra_cerah.ravel(),256,[0,256])

plt.show()
#202331067_Ahmad Miftahul Arza Firisky
```

Output :



Pertama-tamaditentukan nilai sebesar 40, yang berfungsi sebagai faktor penambah kecerahan. Kemudian dibuat array kosong `citra_cerah` yang berukuran sama dengan gambar asli dan memiliki 3 channel warna (R, G, B). Melalui dua perulangan bersarang (`for`), program membaca setiap piksel (`x,y`) pada gambar asli `rgb`, lalu menambahkan nilai `beta` ke setiap komponen warnanya. Hasilnya disimpan ke dalam array `citra_cerah`. Setelah seluruh piksel diproses, array tersebut dikonversi ke dalam format `uint8`, karena tipe data ini adalah standar untuk representasi gambar (dengan rentang nilai 0–255).

- `axs[0,0].imshow(rgb)` untuk menampilkan gambar asli dalam format RGB di posisi kiri atas.
- `axs[0,1].hist(rgb.ravel(),256, [0,256])` untuk menampilkan histogram dari gambar asli. `.ravel()` digunakan untuk meratakan array RGB menjadi satu dimensi, agar bisa dihitung histogramnya secara menyeluruh.

- `axs[1,0].imshow(citra_cerah)` untuk menampilkan gambar hasil setelah cerah (setelah ditambah nilai beta).
- `axs[1,1].hist(citra_cerah.ravel(),256,[0,256])` untuk menampilkan histogram dari gambar yang sudah cerah.

6. Operasi Piksel Untuk Meregangkan Kontras

Operasi Piksel Untuk Meregangkan Kontras

```
[14]: alpha = 1.2
      citra_kontras = np.zeros((baris, kolom, 3))

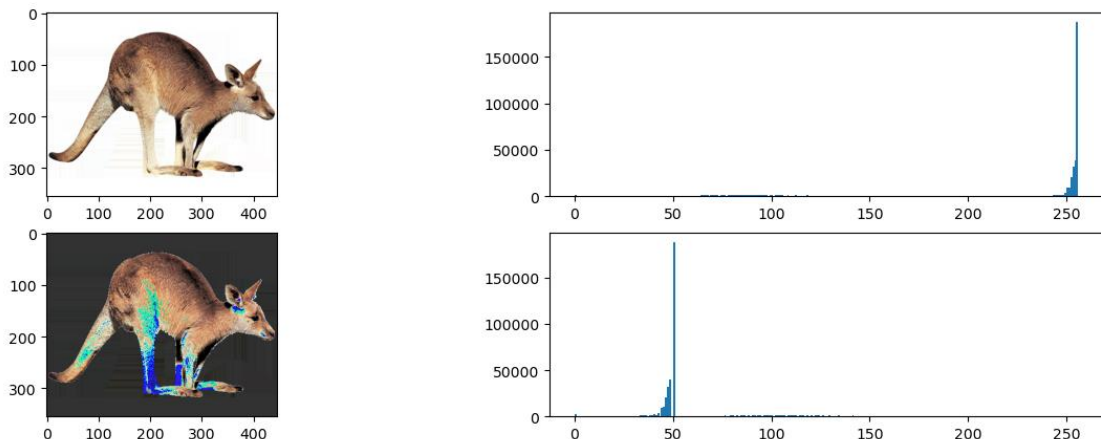
      for x in range(baris):
          for y in range(kolom):
              gyx = rgb[x,y] * alpha
              citra_kontras[x,y] = gyx

      citra_kontras = citra_kontras.astype(np.uint8)

      fig, axs = plt.subplots(2,2, figsize=(15,5))
      axs[0,0].imshow(rgb)
      axs[0,1].hist(rgb.ravel(),256,[0,256])
      axs[1,0].imshow(citra_kontras)
      axs[1,1].hist(citra_kontras.ravel(),256,[0,256])

      plt.show()
      #202331067_Ahmad Miftahul Arza Firisky
```

Output :



`alpha = 1.2` untuk Menetapkan nilai pengali kontras sebesar 1.2. Ini berarti setiap nilai piksel akan dinaikkan sebesar 20%. `citra_kontras = np.zeros((baris, kolom, 3))` untuk Membuat array kosong (berisi nol) dengan ukuran yang sama seperti gambar input dan 3 channel (RGB). `citra_kontras[x,y] = gyx` digunakan untuk Menyimpan nilai hasil konversi ke array baru `citra_kontras`. `citra_kontras = citra_kontras.astype(np.uint8)` untuk Mengubah tipe data array `citra_kontras` ke `uint8` agar sesuai dengan format gambar standar (nilai 0–255). `fig, axs = plt.subplots(2,2, figsize=(15,5))` untuk Membuat plot 2x2 menggunakan Matplotlib untuk menampilkan gambar dan histogram.

- `axs[0,0].imshow(rgb)` untuk Menampilkan gambar asli.
- `axs[0,1].hist(rgb.ravel(),256,[0,256])` untuk Menampilkan histogram gambar asli.
- `axs[1,0].imshow(citra_kontras)` untuk Menampilkan gambar hasil peningkatan kontras.

- `axs[1,1].hist(citra_kontras.ravel(),256,[0,256])` untuk Menampilkan histogram dari gambar hasil peningkatan kontras.

`plt.show()` Menampilkan semua gambar dan histogram pada satu tampilan.

7. Operasi Gabungan Piksel Untuk Antara Meningkatkan Kecerahan dan Meregangkan Kontras

Operasi Piksel Gabungan Antara Meningkatkan Kecerahan dan Meregangkan Kontras

```
[16]: beta = 10
alpha = 1.2
citra_hasil = np.zeros((baris, kolom, 3))

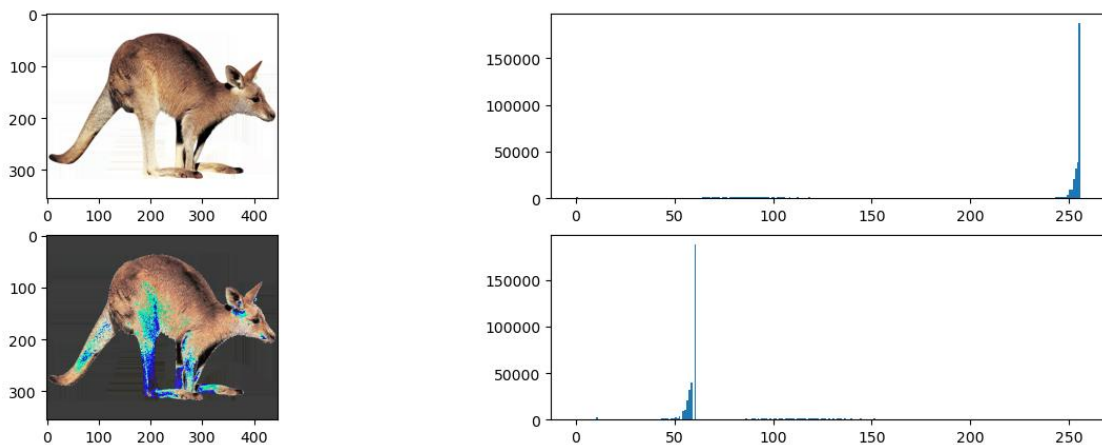
for x in range(baris):
    for y in range(kolom):
        gx = alpha * rgb[x,y] + beta
        citra_hasil[x,y] = gx

citra_hasil = citra_hasil.astype(np.uint8)

fig, axs = plt.subplots(2,2, figsize=(15,5))
axs[0,0].imshow(rgb)
axs[0,1].hist(rgb.ravel(),256,[0,256])
axs[1,0].imshow(citra_hasil)
axs[1,1].hist(citra_hasil.ravel(),256,[0,256])

plt.show()
#202331067_Ahmad Miftahul Arza Firisky
```

Output :



Ini berfungsi untuk melakukan peningkatan kontras dan kecerahan pada sebuah citra berwarna (RGB). Dua parameter digunakan dalam proses ini, yaitu α dan β . Nilai $\alpha = 1.2$ digunakan untuk mengalikan (meningkatkan) kontras, sedangkan $\beta = 10$ berfungsi untuk menambahkan nilai pada tiap piksel sehingga meningkatkan kecerahan.

Pertama, array kosong `citra_hasil` dibuat dengan ukuran yang sama seperti citra asli (`rgb`) dan tiga saluran warna (RGB). Kemudian dilakukan dua lapis perulangan untuk mengakses tiap piksel di koordinat (x, y). Di dalam perulangan, setiap nilai piksel dikalikan dengan α , lalu ditambahkan dengan β , dan hasilnya disimpan ke dalam array `citra_hasil`.

8. Membalik Citra

Membalik Citra

```
[18]: beta = 10
alpha = 1.2
citra_negatif = np.zeros((baris, kolom, 3))

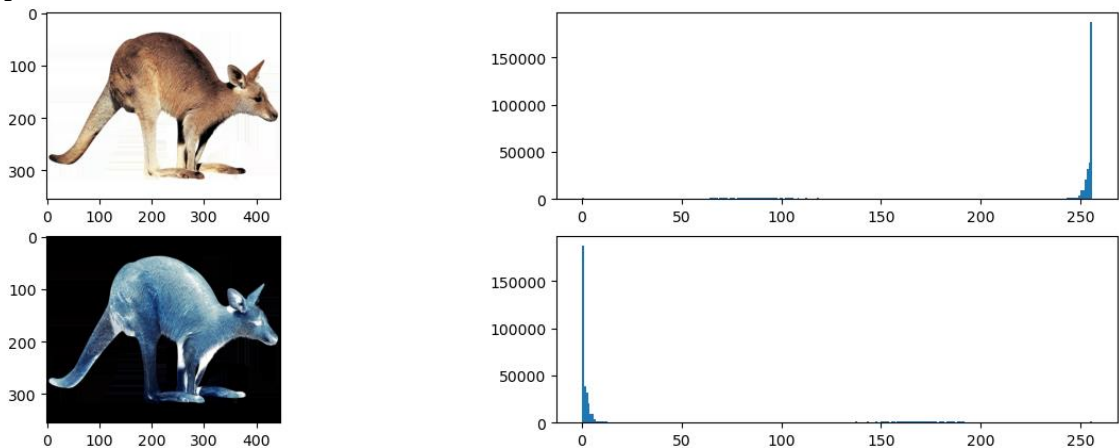
for x in range(baris):
    for y in range(kolom):
        fyx = 255 - rgb[x,y]
        citra_negatif[x,y] = fyx

citra_negatif = citra_negatif.astype(np.uint8)

fig, axs = plt.subplots(2,2, figsize=(15,5))
axs[0,0].imshow(rgb)
axs[0,1].hist(rgb.ravel(), 256, [0,256])
axs[1,0].imshow(citra_negatif)
axs[1,1].hist(citra_negatif.ravel(), 256, [0,256])

plt.show()
#202331067_Ahmad Miftahul Arza Firisky
```

Output :



- `citra_negatif = np.zeros((baris, kolom, 3))` untuk Membuat array kosong untuk menyimpan hasil citra negatif. Ukurannya sama dengan citra input (dengan 3 channel RGB).
- `for x dan y` digunakan untuk perulangan setiap baris piksel citra.
- `fyx = 255 - rgb[x,y]` Mengubah setiap piksel menjadi versi negatifnya dengan cara mengurangi setiap nilai warna dari 255. Proses ini berlaku untuk semua channel warna (R, G, B).
- `citra_negatif[x,y] = fyx` Menyimpan hasil transformasi negatif pada posisi yang sama dalam array `citra_negatif`. `citra_negatif = citra_negatif.astype(np.uint8)` Mengubah tipe data array hasil ke `uint8` agar citra bisa ditampilkan secara benar dalam format gambar standar.

9. Menampilkan Hasil

Menampilkan Hasil

```
[20]: fig, axs = plt.subplots(5,2, figsize=(15,15))

axs[0,0].imshow(rgb)
axs[0,1].hist(rgb.ravel(),256,[0,256])

axs[1,0].imshow(citra_cerah)
axs[1,1].hist(citra_cerah.ravel(),256,[0,256])

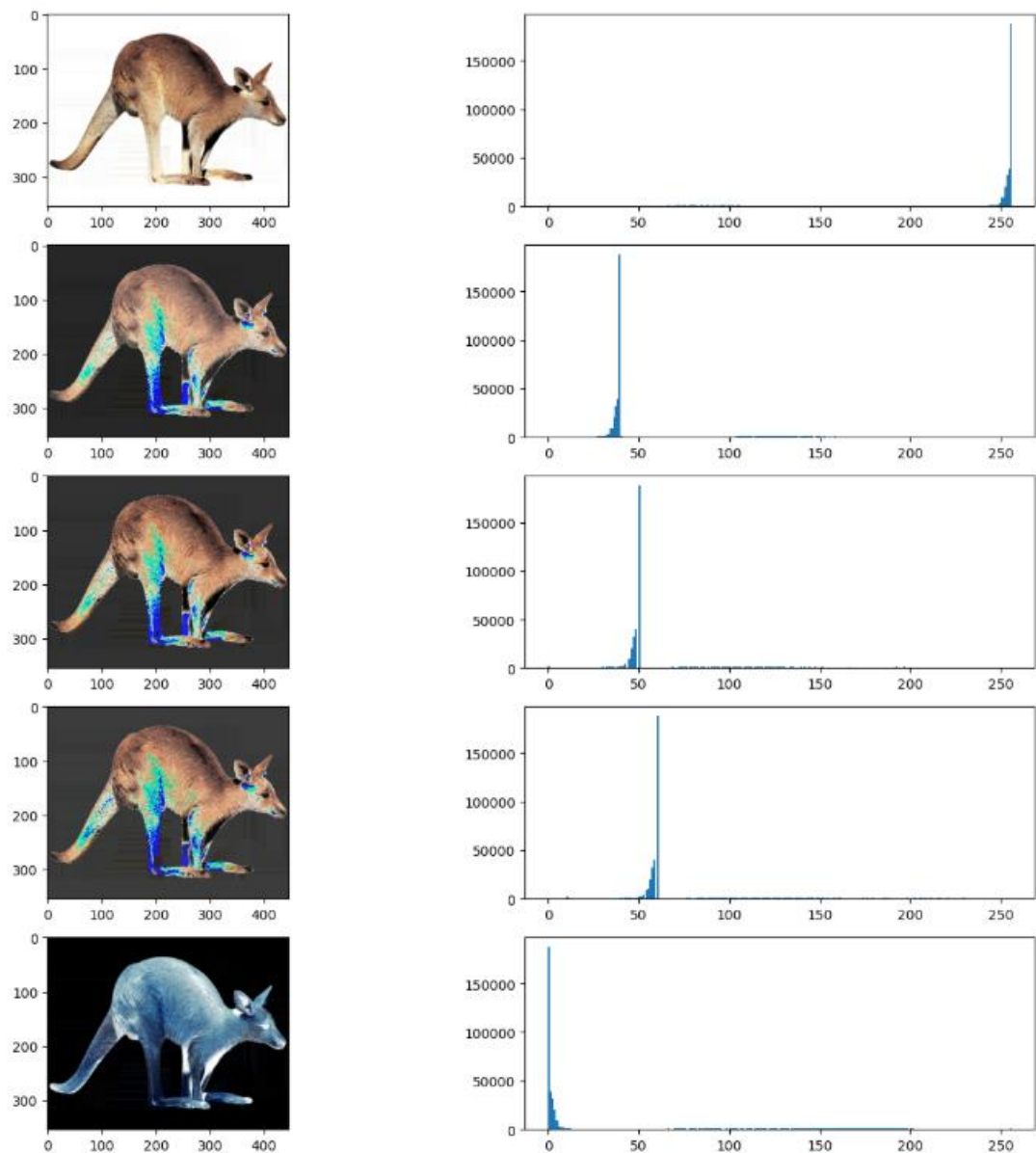
axs[2,0].imshow(citra_kontras)
axs[2,1].hist(citra_kontras.ravel(),256,[0,256])

axs[3,0].imshow(citra_hasil)
axs[3,1].hist(citra_hasil.ravel(),256,[0,256])

axs[4,0].imshow(citra_negatif)
axs[4,1].hist(citra_negatif.ravel(), 256, [0,256])

plt.show()
#202331067_Ahmad Miftahul Arza Firisky
```

Output :



Pertama buat tampilan berukuran 5 baris $\times$ 2 kolom (`fig, axs = plt.subplots(5,2, figsize=(15,15))`) untuk menampung 10 grafik: 5 gambar dan 5 histogram. Baris pertama menampilkan citra asli (`rgb`) dan histogramnya, yang menunjukkan distribusi intensitas warna asli.

- Baris berisi citra yang telah ditingkatkan kecerahannya (`citra_cerah`), bersama histogram yang biasanya akan bergeser ke kanan karena piksel menjadi lebih terang.
- Baris yang menunjukkan citra hasil peningkatan kontras (`citra_kontras`), di mana histogramnya akan lebih tersebar karena rentang nilai piksel diperlebar.
- Baris yang memperlihatkan hasil gabungan antara peningkatan kontras dan penambahan kecerahan (`citra_hasil`), dan histogramnya mencerminkan efek dari kedua transformasi tersebut.
- Terakhir, baris kelima menampilkan citra negatif (`citra_negatif`), dengan histogram yang biasanya tampak terbalik dari histogram gambar asli.

Seluruh tampilan digabungkan dan ditampilkan secara bersamaan melalui perintah `plt.show()`, sehingga pengguna dapat melihat efek dari masing-masing transformasi secara visual dan statistik dalam satu layer.